

Cartography

Minecraft WebGIS 地图模组完整技术方案 v2.0

栅格底图 + 矢量图层 + 现代编辑器 + 插件生态。
本版重点修正坐标、缓存、metatile、权限、阶段验收和发布模型，使方案形成工程逻辑闭环。

Top-down Raster Basemap

Vector Feature Layers

Metatile Rendering

Versioned Tile Namespace

MC CRS

Plugin SDK

版本: v2.0 逻辑闭环版

日期: 2026-06-24

范围: 从 MVP 到完整平台化版本的产品、渲染、数据、缓存、权限、API、部署和测试方案。

本文档为技术方案草案，不包含商标、品牌、法务和最终工作量评估。所有坐标、缓存、权限和发布约束以本文 v2.0 的不变量为准。

目录

- 0. v2.0 修订摘要
- 1. 项目定位和设计原则
- 2. 阶段化成功标准
- 3. 总体架构和数据流
- 4. 坐标系统和 tile grid
- 5. 渲染系统
- 6. Dirty 更新、metatile 和祖先失效
- 7. 缓存、版本化瓦片和发布模型
- 8. 图层、Feature 和数据模型
- 9. 权限、隐私和安全门禁
- 10. POI 权重和标签引擎
- 11. Web 编辑器
- 12. API 和插件 SDK
- 13. 存储和部署拓扑
- 14. 路线图: MVP 到完全完善
- 15. 测试、观测和验收
- 16. 风险清单和决策点
- 附录 A. 配置、伪代码和协议示例
- 附录 B. 参考资料

0. v2.0 修订摘要

v1 的主线是成立的: Cartography 应该是一个面向 Minecraft 世界的 WebGIS 平台，而不是传统 marker 插件。但 v1 把愿景、MVP、数据坐标、显示坐标、渲染缓存和权限语义混在了一起。v2.0 做的不是改名或补句子，而是把核心不变量重新定义。

v2.0 的五个硬不变量

1. **底图只表达世界外观。** 可点击、可搜索、可授权、可样式切换的对象必须进入 GIS Feature，不允许被 renderer 内置语义特判。
2. **数据坐标永远使用 Minecraft 原始坐标。** Feature 几何存储 X/Y/Z 或 X/Z，不随 spawn、显示 origin 或 tile 发布边界变化。
3. **dirty 更新以 metatile 为调度单位。** block 或 chunk 变化先映射到 max zoom tile，再归并成 metatile job，最后切出普通 tile。
4. **低 zoom 通过祖先失效链维护。** max zoom tile 更新后必须标记所有父级 tile dirty，但允许异步、低优先级刷新。
5. **权限是硬过滤，不是 POI 打分项。** 未授权对象不能进入候选集，更不能靠 hidden penalty 侥幸显示。

评审问题	v2.0 修订方式	影响范围
MVP 成功标准和 MVP 范围不一致	拆成 MVP 成功标准 与 产品级成功标准 ，每一阶段都有独立验收口径。	路线图、验收、产品叙事
底图与业务图层边界模糊	定义 source-of-truth 规则: 外观归 raster，语义归 feature；rail 等既可以作为像素外观，也可以由插件抽取成 vector network，但不能由 renderer 同时承担语义层。	渲染器、插件、编辑器
坐标系统自相矛盾	数据存储只用 MC 原始坐标；显示 origin 和 tileOrigin 只存在于 ViewTransform/TileGrid。	Feature、MVT、导入导出、前端
负坐标 tile 未说明	在线 signed tile 与发布态 normalized tile 分离；PMTiles/CDN 使用非负归一化坐标并在 manifest 中记录偏移。	Tile URL、CDN、PMTiles
普通 tile 调度违背 metatile 原则	dirty chunks -> affected maxZoom tiles -> metatile ids -> enqueue metatile jobs -> cut 256 tiles。	Render scheduler、缓存
低 zoom 更新逻辑不完整	引入 tile invalidation DAG 和 ancestor invalidation。	Tile pyramid、后台任务
所有 block change 都 dirty 过度触发	MVP 允许保守策略，但明确后续 surface-aware dirty filter。	性能、TPS 保护
透明建筑可能消失	把透明对象拆成 thin_overlay、transparent_structure、fluid_surface。玻璃/冰建筑参与结构化表面合成，不再被简单穿透。	Surface extraction、材质系统
GeoJSON 标准表述不严谨	改为 Cartography Feature JSON: 借用 GeoJSON Geometry 结构，但坐标语义为 MC CRS，不声称符合 RFC 7946 地理语义。	导入导出、GIS 兼容性
数据流图箭头错误	重画数据流: render_jobs -> renderer -> tile_cache；permissions -> operations；features/layers -> vector cache invalidation。	架构图、模块依赖
权限隐藏只是 ranking penalty	permission filter 在 ranking 之前执行；ranking 只对已授权候选集排序。	安全、隐私、POI
cache key 与 tile URL 不匹配	tile URL 加 tilesetVersion；tilesetVersion 由 profile、resourcePack、rendererVersion、materialTable 等共同生成。	浏览器缓存、CDN、发布
原子写入外推到对象存储	区分 filesystem atomic rename 和 object storage content-addressed object + manifest switch。	存储、CDN
PMTiles 在线更新边界不清	明确 PMTiles 仅作为 static archive package，不作为在线编辑态主写入目标。	部署、归档
玩家位置隐私默认冲突	引入 serverMode preset 与 location privacy policy，公共地图默认延迟、模糊或关闭。	实时层、安全
Beta/v1 API 边界反复	API 分为 internal、editor、plugin SDK 三层，plugin SDK 到 v1 才冻结。	生态、版本兼容

1. 项目定位和设计原则

Cartography 是一个面向 Minecraft 服务器的 WebGIS 地图平台。它的底层能力是把世界渲染成可浏览的地图瓦片，但真正差异化能力是: 可编辑空间对象、可授权图层、可搜索 POI、可计算标签权重、可接入服务器插件生态。

一句话定位

Cartography 不是“更清楚的地图截图”，而是 Minecraft 世界的空间知识库: 世界外观由 raster basemap 表达，服务器文明、道路、建筑、领地、事件和交通网络由 vector feature layers 表达。

1.1 产品边界

类别	属于 Cartography	不属于第一阶段目标
底图	正射 top-down raster tiles、维度切换、渐进式瓦片金字塔、材质 profile。	完整 3D 客户端渲染、光线追踪级复刻、动态实体模型。
图层	道路、建筑、区域、领地、POI、交通网络、玩家位置、活动热力图。	把所有图层烤进底图图片。
编辑	Web 编辑器、点线面绘制、snap、审核、版本、回滚。	一开始就做复杂 CAD 或专业 ArcGIS 全套功能。
生态	提供 SDK 让插件写入 feature/layer。	让 renderer 直接内置每个插件的业务语义。
部署	单服本地、PostGIS 专业版、对象存储/CDN、PMTiles 静态归档。	把 PMTiles 当在线高频写入数据库。

1.2 核心设计原则

- 外观和语义分离。** 方块长什么样属于底图；它是不是道路、商店、领地、危险区，属于 feature。
- 坐标语义先于瓦片路径。** 数据坐标稳定，显示坐标和瓦片坐标可变，发布态可归一化。
- 渲染永不压过服务器体验。** 主线程只捕获快照；渲染、编码、IO 必须后台化，并受 TPS/CPU/IO budget 约束。
- 缓存命名空间可证明。** 用户看到的 tile URL 必须包含足够版本信息，不能只在内部 metadata 里记 hash。
- 安全默认保守。** 玩家位置、隐藏基地、战争服坐标、权限图层先过滤后排名。
- SDK 晚冻结。** 内部 API 可以快速变化，第三方插件 API 必须等数据模型稳定后冻结。

2. 阶段化成功标准

v2.0 不再把愿景指标放进 MVP 验收。MVP 的职责是证明底图渲染、增量更新、缓存和浏览体验成立；产品级目标由 Alpha、Beta、v1 和后续版本逐步承担。

2.1 成功标准分层

层级	成功标准	不要求
MVP	稳定渲染正射底图；dirty chunk 增量更新；可浏览、多维度、玩家 marker；TPS 保护可观测；瓦片金字塔可用。	点线面编辑、审核、完整样式系统、插件 SDK、MVT。
Alpha	空间对象能被维护：点、线、面 feature；图层权限；草稿/审核；版本历史；基本搜索。	十万级 feature、复杂标签避让、第三方 SDK 冻结。
Beta	矢量瓦片、样式文档、POI 权重、标签碰撞、导入导出、坐标/缓存模型稳定。	大量插件生态和长期兼容承诺。
v1	Plugin SDK 冻结；核心插件集成；生产部署文档；PostGIS 可选；对象存储发布；稳定 API。	影院级离线渲染和完整 3D。
v2+	高清材质、多渲染 profile、静态归档、分析图层、治理和协作增强。	无边界地兼容所有 modded 动态模型。

2.2 MVP 验收清单

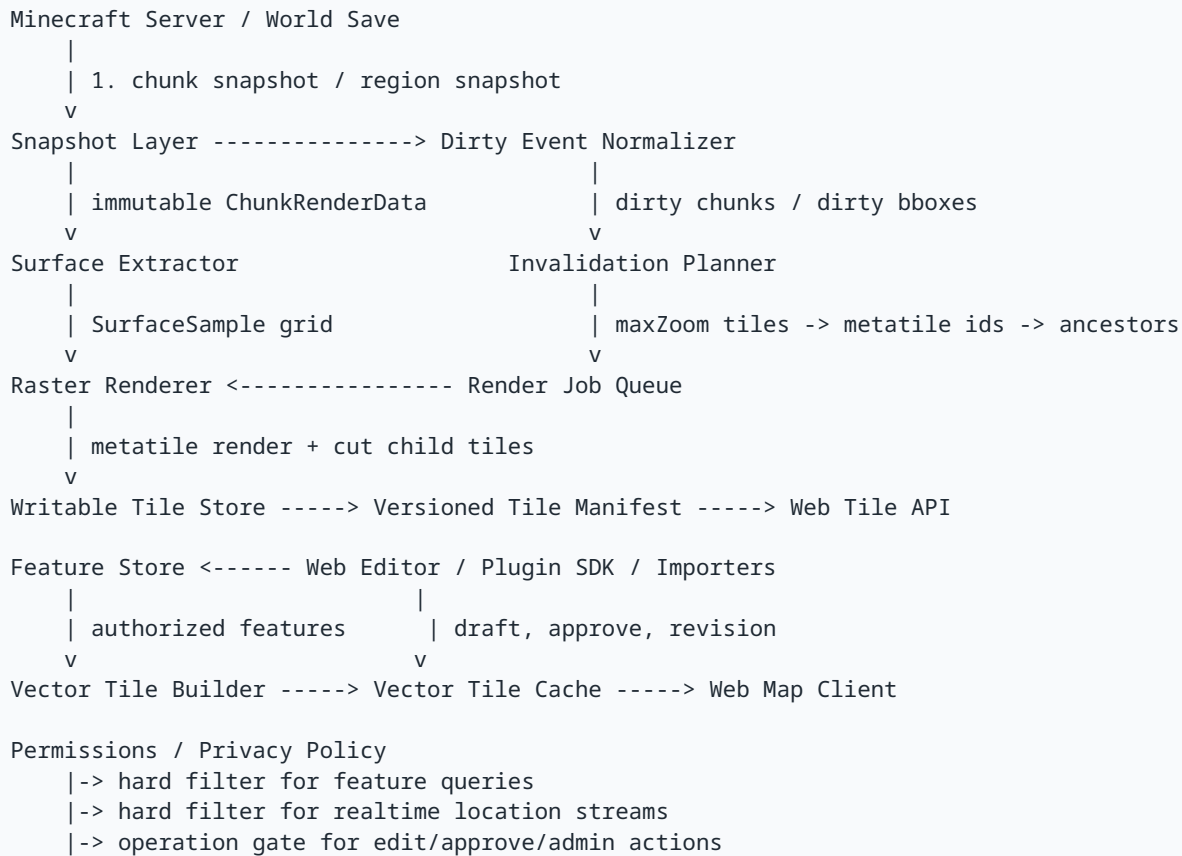
- 在 Paper 单服或 Fabric 单服上渲染 overworld/nether/end 的 top-down 底图。
- 首次渲染后，前端可以按 zoom/pan 流畅浏览瓦片。
- block/chunk 变化通过 dirty pipeline 更新最高 zoom tile，并异步刷新父级低 zoom tile。
- 渲染任务不会在服务器 TPS 低于阈值时继续抢资源。
- 所有 tile URL 使用版本化 tileset namespace，切换材质包或渲染器版本不会污染旧缓存。
- 坐标 round-trip 测试通过: MC 原始坐标 -> view pixel -> signed tile -> normalized publish tile -> MC 原始坐标。

2.3 产品级验收清单

- 玩家或管理员能在地图上维护道路、建筑、区域、POI 等空间对象。
- 不同用户看到的 feature 集合由权限硬过滤决定。
- 缩放时根据类型、重要性、空间上下文和标签冲突显示合适的信息密度。
- 第三方插件可以通过稳定 SDK 写入图层，且不会依赖 renderer 内部实现。
- 可导出赛季静态地图，并明确 PMTiles 只用于归档/发布，不用于在线高频写入。

3. 总体架构和数据流

Cartography 的架构按照“世界数据采集、底图渲染、空间数据、发布缓存、前端编辑”分层。关键是数据流方向必须明确: render job 生成 tile cache; feature/layer 变更生成 vector cache invalidation; permissions 只约束操作和查询, 不是渲染任务的下游产物。



3.1 模块边界

模块	输入	输出	边界约束
Server Adapter	Paper/Fabric/NeoForge API、world save。	ChunkSnapshot 或 ChunkRenderData。	只做短生命周期采集，不做图片编码和大 IO。
Surface Extractor	ChunkRenderData、材质规则、dimension surface rule。	SurfaceSample grid、heightmap。	不产生业务语义 feature。
Raster Renderer	SurfaceSample grid、render profile、texture atlas。	metatile image、切割后的 raster tiles。	只表达世界外观。
Feature Store	编辑器、插件、导入器写入的空间对象。	Feature、revision、audit、权限绑定。	保存 MC 原始坐标，不保存显示偏移坐标。
Vector Tile Builder	Feature Store 查询结果。	MVT 或 GeoJSON tile。	先权限过滤，再样式/排名/碰撞。
Tile Manifest	tilesetVersion、profile、resourcePackHash、rendererVersion。	发布态 tile namespace。	浏览器/CDN 依据 URL 版本化缓存。
Web Client	raster tiles、vector tiles、realtime streams。	浏览、编辑、查询、样式。	不能绕过 API 权限过滤直接拿隐藏 feature。

3.2 三类数据源

Raster Basemap

由 MC 世界外观渲染出来的栅格瓦片。它是背景，负责让人看懂地形、建筑、水体、植被和光照。

Vector Features

由用户、管理员、导入器或插件维护的结构化空间对象。它们可点击、可搜索、可授权、可回滚、可样式化。

Realtime Overlays

玩家位置、在线事件、临时战斗区等实时流。它们通常不进入长期 feature 历史，且必须有隐私策略。

Computed Layers

热力图、活动密度、道路中心性、旅行时间等由任务计算生成的派生图层。它们有自己的 source version。

4. 坐标系统和 tile grid

坐标系统是 Cartography 的地基。v2.0 的原则是: **数据坐标稳定，显示坐标可变，发布坐标可归一化**。不能让 spawn、tile origin 或 CDN 约束污染 feature 存储语义。

4.1 Canonical MC CRS

Cartography 定义内部坐标参考为 `cartography:mc-crs`。它不是真实地球 CRS，不是 EPSG:4326，也不声称符合 RFC 7946 的 GeoJSON 地理语义。RFC 7946 的 GeoJSON 默认使用 WGS 84 和十进制度数，Minecraft X/Z 不能被 GIS 工具直接当作经纬度使用 [R3]。

```
Canonical data coordinate:
world_id      = stable world identifier
dimension_id  = overworld | nether | end | custom
x             = Minecraft block X, integer or floating point
y             = Minecraft block Y, optional for 2D features
z             = Minecraft block Z, integer or floating point

2D feature geometry uses:
coordinate tuple = [x, z]

3D or height-aware feature may use:
coordinate tuple = [x, y, z]
or properties.minY / properties.maxY for extruded objects
```

重要规则

Feature geometry 永远不做 `x - originX` 存储。origin 只存在于 ViewTransform 和 TileGrid 中。这样 spawn 改变、地图重发布、PMTiles 归一化或前端切换地图中心，都不会使老数据整体偏移。

4.2 ViewTransform 与 TileGrid 分离

```
Data coordinate:
  dataX = minecraftX
  dataZ = minecraftZ

View transform, only for screen rendering:
  viewX = dataX - viewOriginX
  viewY = dataZ - viewOriginZ

Max zoom pixel coordinate:
  pixelX = (dataX - tileOriginX) * pixelsPerBlock
  pixelY = (dataZ - tileOriginZ) * pixelsPerBlock

Signed tile coordinate:
  signedTileX = floor(pixelX / tileSize)
  signedTileY = floor(pixelY / tileSize)
```

4.3 负坐标处理

Minecraft 世界天然存在负 X 和负 Z。Cartography 支持两种 tile coordinate 模式，不把它们混在一起。

模式	用途	tile 坐标	URL/存储方式
Online signed grid	在线可写 tile store、开发环境、本地文件系统。	允许 signedTileX/signedTileY 为负。	路径显式编码符号，例如 <code>/z8/x-3/y12.webp</code> ，避免路径解析歧义。
Published normalized grid	CDN、对象存储、PMTiles、静态归档。	非负 normTileX/normTileY。	manifest 记录 <code>minSignedTileX</code> 与 <code>minSignedTileY</code> ，发布路径只使用非负整数。

```
Normalization for published tileset:
  normTileX = signedTileX - minSignedTileX
  normTileY = signedTileY - minSignedTileY

Reverse mapping:
  signedTileX = normTileX + minSignedTileX
  signedTileY = normTileY + minSignedTileY

The published tileset manifest must include:
  minSignedTileX, minSignedTileY,
  maxSignedTileX, maxSignedTileY,
  tileOriginX, tileOriginZ,
  pixelsPerBlock, tileSize, maxZoom, minZoom
```

4.4 前端 TileGrid 声明

OpenLayers 适合自定义投影和本地坐标系，因为它的 Projection API 支持自定义投影、extent、units 等定义 [R8]。MapLibre 也可以工作，但要通过 style/source 和自定义 tile scheme 映射到 MC CRS。

CartographyTileGrid manifest example:

```
{
  "crs": "cartography:mc-crs",
  "world": "survival_2026",
  "dimension": "overworld",
  "dataCoordinate": "minecraft-xz",
  "tileCoordinateMode": "published-normalized",
  "tileOrigin": { "x": 0, "z": 0 },
  "signedExtent": { "minX": -183, "minY": -96, "maxX": 220, "maxY": 144 },
  "normalizedOffset": { "x": 183, "y": 96 },
  "tileSize": 256,
  "maxZoom": 8,
  "pixelsPerBlockAtMaxZoom": 4
}
```

5. 渲染系统

第一版渲染器只做正射 top-down raster basemap。斜视图、完整 3D、复杂 block entity 渲染都不进入 MVP。Paper 的 ChunkSnapshot 是静态、线程安全快照，设计目的包括把 chunk 数据交给另一个线程处理，例如地图渲染 [R1]；这正好支持主线程快照、后台渲染的分层。

5.1 渲染管线

```
Server Adapter
-> ChunkSnapshot / RegionSnapshot
-> ChunkRenderData
-> SurfaceSample grid
-> Material resolve / texture atlas
-> height shading / biome tint / fluid compositing
-> metatile raster image
-> cut to 256 tiles
-> write versioned tile store
-> schedule ancestor downsample
```

5.2 SurfaceSample

```
record SurfaceSample(
    int worldX,
    int worldZ,
    int surfaceY,
    String blockId,
    String blockStateKey,
    String biomeId,
    int skyLight,
    int blockLight,
    SurfaceKind kind,
    List<OverlaySample> overlays,
    int waterDepth,
    int materialColor,
    int heightForShading
) {}

enum SurfaceKind {
    OPAQUE_SURFACE,
    TRANSPARENT_STRUCTURE,
    THIN_OVERLAY,
    FLUID_SURFACE,
    FOLIAGE_SURFACE,
    DECORATION_VISUAL,
    UNKNOWN_FALLBACK
}
```

5.3 透明对象分类

v1 把 glass、ice、水面、雪层等都放进 transparent/overlay，容易让玻璃建筑被穿透掉。v2.0 改为按视觉语义分类。

类别	例子	表面提取策略	是否可穿透
thin_overlay	雪层、地毯、藤蔓、苔藓覆盖。	作为 overlay 合成到下方 base。	通常可穿透。
transparent_structure	玻璃、染色玻璃、冰、玻璃板、温室顶部。	作为结构表面参与渲染，必要时和下方颜色 alpha blend。	不能简单忽略。
fluid_surface	水、熔岩。	先定位流体表面，再采样下方底色和深度。	按 fluid 规则穿透。
decoration_visual	火把、花、铁轨方块外观、按钮。	高 zoom 可作为像素细节参与底图，但不产生可点击语义。	通常不阻挡结构表面。
opaque_surface	石头、木板、泥土、屋顶、道路方块。	作为基础表面。	不可穿透。

5.4 底图和结构化图层边界

rail / road 的三种身份必须分开

铁轨方块作为世界外观时，只是 raster basemap 的像素；铁路网络作为可搜索、可点击、可授权、可样式切换的交通系统时，必须是 vector feature；高 zoom 图标若只是视觉增强，不能承载业务语义。

对象	底图表达	Feature 表达	谁负责
铁轨方块	像素纹理或颜色。	无。	Raster Renderer。
铁路交通线	不烤进底图。	LineString + station POI + route properties。	Rail integration plugin / SDK。
道路方块	地面外观。	无。	Raster Renderer。
道路网络	不烤进底图。	LineString + road class + name + permissions。	Editor 或 roads plugin。
火把/花	高 zoom 可作为视觉细节。	通常无，除非插件创建 POI。	Renderer 只画外观。

5.5 渲染 profile

Profile	用途	特性	适合阶段
fast	MVP 在线增量。	颜色表、biome tint、基础 height shading、WebP。	MVP
vanilla	生产默认。	top face texture、透明结构、水深、柔和 slope shade。	Beta/v1
hd	高清展示。	4-8 pixels/block、材质采样、AO 近似、软阴影。	v2+
archive	赛季归档。	离线高质量重渲染、可生成 PMTiles。	v2+

```
MVP render profile:
renderer:
  projection: top_down
  tileSize: 256
  maxZoom: 8
  minZoom: 0
  pixelsPerBlockAtMaxZoom: 2
  format: webp
  quality: 85
surface:
  transparentStructure: preserve
  thinOverlay: composite
  fluidDepth: true
lighting:
  heightShade: true
  softShadow: false
performance:
  metatileSize: 4
  paddingBlocks: 2
  pauseBelowTps: 18.5
```

6. Dirty 更新、metatile 和祖先失效

Cartography 采用“保守可用，逐步精细”的 dirty 更新策略。MVP 可以把任何 block change 标为 chunk dirty，但方案必须承认这会产生无效渲染，并定义后续的 surface-aware filter。

6.1 正确的数据流

```
Block / Chunk Event
-> dirty chunk set
-> debounce and merge
-> dirty bbox in MC coordinates
-> affected maxZoom signed tiles
-> affected metatile ids
-> enqueue metatile render jobs
-> render metatile with padding
-> cut 256x256 child tiles
-> mark ancestors dirty
-> async downsample lower zoom tiles
```

6.2 Dirty chunk 到 metatile

```
function onBlockChanged(world, dimension, x, y, z, blockBefore, blockAfter):
  cx = floorDiv(x, 16)
  cz = floorDiv(z, 16)

  if surfaceAwareFilterEnabled:
    surfaceY = surfaceIndex.lookup(world, dimension, x, z)
    if y < surfaceY and not mayAffectSurface(blockBefore, blockAfter):
      markLowPriorityAuditOnly(cx, cz)
      return

  dirtyChunks.add(world, dimension, cx, cz)

function planRenderJobs(dirtyChunks):
  bboxes = mergeChunksToBBoxes(dirtyChunks)
  for bbox in bboxes:
    padded = bbox.expand(paddingBlocksForProfile)
    tiles = bboxToSignedTiles(padded, maxZoom)
    metatiles = unique(map(tileToMetatileId, tiles))
    enqueueMetatileJobs(metatiles)
    markAncestorsDirty(tiles)
```

6.3 Metatile job

普通 256 tile 不是调度单位。调度单位是 metatile，例如 4x4 个普通 tile 加 padding。这样可以保证阴影、水体合成、抗锯齿和边缘采样在同一次渲染中一致。

```

metatileSize = 4
normalTileSize = 256
metatilePixelSize = metatileSize * normalTileSize
paddingPixels = paddingBlocks * pixelsPerBlockAtMaxZoom

function renderMetatile(job):
    renderBBBox = metatileToWorldBBBox(job.metatileId).expand(paddingBlocks)
    snapshots = snapshotProvider.loadChunks(renderBBBox)
    samples = surfaceExtractor.extract(renderBBBox, snapshots)
    image = rasterRenderer.render(samples, renderBBBox)

    cropped = removePadding(image)
    childTiles = cutIntoTiles(cropped, normalTileSize)
    tileWriter.writeBatch(childTiles, job.tilesetVersion)

```

6.4 祖先失效链

最高 zoom tile 改了以后，所有父级 tile 都要标 dirty。低 zoom 不需要同步更新，但必须有明确的 invalidation DAG，否则缩小地图可能长期显示旧世界。

```

function markAncestorsDirty(maxZoomTiles):
    for tile in maxZoomTiles:
        z = tile.z
        x = tile.x
        y = tile.y
        while z > minZoom:
            parent = TileCoord(z - 1, floorDiv(x, 2), floorDiv(y, 2))
            lowZoomDirtySet.add(parent)
            z = parent.z
            x = parent.x
            y = parent.y

function refreshLowZoom():
    # low priority, batch-friendly
    while budgetAvailable():
        parent = lowZoomDirtySet.popBest()
        children = loadChildren(parent)
        image = downsample(children)
        writeTile(parent, image)
        markParentDirtyIfNeeded(parent)

```


6.5 Surface-aware dirty filter

事件	MVP 策略	后续优化
地表方块变化	dirty chunk。	立即 maxZoom metatile。
地下挖矿	dirty chunk，保守。	若 $y < \text{surfaceY}$ 且不影响光/流体/暴露面，则 skip 或低优先级。
水/熔岩变化	dirty chunk。	即使低于 surfaceY，也可能改变流体表面或发光，保留。
光源变化	dirty chunk。	只在启用 blockLight 渲染时触发。
树叶/玻璃/雪层变化	dirty chunk。	按 SurfaceKind 决定是否影响 top surface。
大批量 WorldEdit	dirty region。	事件合并、debounce、区域优先级降噪。

7. 缓存、版本化瓦片和发布模型

识别 cache key 不够，URL namespace 必须也能反映版本。否则即使内部 metadata 记录了 resourcePackHash，浏览器和 CDN 仍可能拿旧 URL 下的旧 tile。

7.1 tilesetVersion

```
tilesetVersion = sha256(
    worldId + dimensionId +
    renderProfileId + renderProfileHash +
    resourcePackHash + materialTableHash +
    rendererVersion + rendererAlgorithmHash +
    tileGridVersion +
    format + quality
).short(16)
```

```
Raster tile URL:
/tiles/raster/{world}/{dimension}/{profile}/{tilesetVersion}/{z}/x{x}/y{y}.webp

Examples, online signed grid:
/tiles/raster/survival/overworld/fast/a18f03c99f2e10b7/8/x-3/y12.webp

Examples, published normalized grid:
/tiles/raster/survival/overworld/fast/a18f03c99f2e10b7/8/x180/y108.webp
```

7.2 文件系统和对象存储写入语义

存储	写入方式	原子性模型	适合场景
Filesystem	写临时文件 -> fsync -> rename -> 同批 metadata。	POSIX rename 在同文件系统内可近似原子。	单服、MVP、本地缓存。
Object Storage	按内容 hash 上传 object -> 更新 manifest 指针。	对象没有 POSIX rename；通过 manifest 切换发布版本。	CDN、分布式发布。
PMTiles	离线构建完整 archive。	只读归档，不能原地事务更新。	赛季归档、静态站点。

```

Filesystem writer, batch-aware:
  write tile.tmp
  fsync tile.tmp
  rename tile.tmp -> tile.webp
  write metadata.tmp
  fsync metadata.tmp
  rename metadata.tmp -> metadata.json

Object storage writer:
  contentKey = /objects/{sha256(tileBytes)}.webp
  PUT contentKey
  update draft manifest: tileCoord -> contentKey
  when batch complete:
    publish manifestVersion atomically in database
    CDN URL points to immutable tilesetVersion or manifestVersion

```

7.3 Tile store 角色

角色	推荐实现	是否在线写入	说明
online writable raster store	filesystem / SQLite blob / object storage manifest。	是。	承载当前服务器在线更新。
online vector query store	SQLite / PostGIS。	是。	承载 feature 查询、权限过滤、MVT 生成。
static archive package	PMTiles。	否。	适合导出赛季地图、展览地图、只读发布。PMTiles 文档明确它是 read-only，不能原地更新 [R7]。

7.4 HTTP 缓存策略

- 含 `tilesetVersion` 的 tile URL 可使用长缓存: `Cache-Control: public, max-age=31536000, immutable`。
- manifest URL 使用短缓存或 ETag: `Cache-Control: no-cache`。
- 在线 signed grid 可短缓存；发布态 normalized grid 可长缓存。
- 换材质包、renderer、profile、tile grid 时生成新 `tilesetVersion`，不覆盖旧 namespace。

8. 图层、Feature 和数据模型

Cartography 的结构化数据不直接使用标准 GeoJSON 地理语义，而是使用 **Cartography Feature JSON**: 借用 GeoJSON 的 Geometry 形状表达，但坐标解释为 `cartography:mc-crs`。这样避免 GIS 工具把 X/Z 误解为经纬度。

8.1 Cartography Feature JSON

```
{
  "type": "CartographyFeatureCollection",
  "crs": {
    "type": "cartography:mc-crs",
    "world": "survival_2026",
    "dimension": "overworld",
    "axis": "x,z",
    "unit": "minecraft_block"
  },
  "features": [
    {
      "type": "CartographyFeature",
      "id": "building_spawn_market",
      "layer": "buildings",
      "geometry": {
        "type": "Polygon",
        "coordinates": [[[120, 80], [180, 80], [180, 140], [120, 140], [120, 80]]]
      },
      "properties": {
        "name": "中央市场",
        "category": "commerce",
        "importance": 85,
        "owner": "Steve"
      },
      "visibility": {
        "minZoom": 5,
        "maxZoom": 18
      }
    }
  ]
}
```

8.2 数据库表

表	职责	关键字段
carto_worlds	世界和维度注册。	world_id, dimension_id, min_y, max_y, server_mode
carto_tile_grids	tile grid / coordinate manifest。	grid_id, tile_origin_x, tile_origin_z, signed_extent, normalized_offset
carto_tilesets	版本化底图发布单元。	tileset_version, profile_hash, resource_pack_hash, renderer_version, state
carto_render_jobs	metatile 渲染任务。	job_id, metatile_id, priority, state, tileset_version
carto_tile_manifest	object storage 或发布态 tile 映射。	tileset_version, z, x, y, content_key, etag
carto_layers	图层定义。	layer_id, world_id, dimension_id, source_type, style_json, permission_policy_id
carto_features	当前 feature 状态。	feature_id, layer_id, geometry, bbox, properties_json, weight, revision
carto_feature_revisions	历史版本。	feature_id, revision, author, diff_json, created_at
carto_permissions	权限策略。	policy_id, subject, action, condition_json
carto_audit_log	审计和回滚。	actor, action, target, before, after, timestamp

8.3 Feature 层级

层级	例子	持久性	是否进入 MVT
base feature	道路、建筑、区域、领地、POI。	长期。	是。
computed feature	热力区、旅行时间等值线、活动密度。	任务生成，可重算。	是。
draft feature	玩家提交的建筑边界草稿。	审核前可见范围受限。	只向有权限用户输出。
realtime entity	玩家位置、临时事件。	短期或不持久。	通常走 realtime API，不进入长期 MVT。

9. 权限、隐私和安全门禁

权限不能靠 POI 分数“扣一扣”。v2.0 将权限处理放在任何排序、聚类、样式计算之前。未授权 feature 不进入候选集，也不会被计入标签冲突和聚类。

9.1 查询流水线

Request context

-> authenticate user / role / server mode

-> permission hard filter

-> spatial query by bbox / tile / viewport

-> style visibility filter

-> ranking and label collision

-> serialize vector tile / JSON response

```
function queryVisibleFeatures(context, bbox, layers):
  allowedLayers = permissionEngine.allowedLayers(context, "view", layers)
  rawCandidates = featureStore.query(bbox, allowedLayers)
  authorized = []
  for feature in rawCandidates:
    if permissionEngine.canView(context, feature):
      authorized.append(feature)
  styled = styleEngine.applyVisibility(context.zoom, authorized)
  ranked = labelEngine.rankAndDeclutter(styled)
  return ranked
```

9.2 权限类型

权限	含义	例子
view	能否看到 feature 或图层。	隐藏基地、战争情报、管理员注记。
edit	能否编辑 feature。	玩家只能编辑自己提交的道路草稿。
approve	能否把草稿发布为正式图层。	cartographer/admin 审核建筑边界。
manage	能否改图层样式、权限、导入导出。	地图管理员。
realtime.view	能否看实时实体流。	队友位置、管理员精确位置。

9.3 玩家位置隐私

玩家位置是 P0 能力，但默认行为必须由服务器模式决定。生存、PVP、战争服的公共地图默认不暴露所有玩家精确坐标。

服务器模式	公共地图默认	好友/队友	管理员
building	可显示在线玩家，允许精确或短延迟。	精确。	精确。
pve	默认延迟 30-120 秒，可关闭。	精确或短延迟。	精确。
survival	默认模糊到 chunk/region 或关闭。	经授权后显示。	精确。
pvp	默认关闭公共精确位置。	队伍内可选。	精确。
war	默认关闭，支持战争迷雾。	按阵营策略。	精确。

10. POI 权重和标签引擎

标签引擎只处理已授权候选集。它解决的是“在当前 zoom、viewport、样式和用户上下文下，哪些对象应该显示以及如何避免互相糊脸”。

10.1 候选集流水线

```
visibleCandidates = PermissionFilter(context, features)
styledCandidates  = StyleVisibilityFilter(zoom, visibleCandidates)
rankedCandidates  = RankingEngine.score(styledCandidates)
placedLabels      = CollisionEngine.place(rankedCandidates)
serializedOutput   = VectorTileEncoder.encode(placedLabels)
```

10.2 评分公式

```
score =
    base_importance
    * type_weight
    * zoom_factor
    * user_context_factor
    * freshness_factor
    * spatial_context_factor
    * semantic_density_factor
    - clutter_penalty

# 不包含 permission penalty。
# 权限在候选集之前硬过滤。
```

因子	说明	例子
base_importance	管理员或插件给定的重要性。	主城 100，普通农场 30。
type_weight	不同类型的默认权重。	spawn、交通枢纽、主城高于小型建筑。
zoom_factor	不同 zoom 的显示偏好。	低 zoom 显示城市/区域，高 zoom 显示店铺/入口。
user_context_factor	用户角色和上下文。	居民看到本城设施更高，游客看到公共交通更高。
freshness_factor	最近活动或新建筑。	新开放市场短期提权。
spatial_context_factor	空间位置重要性。	路口、区域中心、传送门附近。
clutter_penalty	同屏标签过密惩罚。	候选标签碰撞时低分者隐藏或聚类。

10.3 Zoom 策略

Zoom	显示策略	典型对象
低 zoom	抽象世界结构。	主城、区域名、主干道、主要交通枢纽。
中 zoom	展开道路和建筑群。	道路名、商区、铁路站点、主要建筑边界。
高 zoom	显示细节和交互入口。	店铺、农场入口、NPC、局部标注、玩家草稿。

11. Web 编辑器

编辑器是 Cartography 和普通网页地图插件的分水岭。MVP 不要求编辑器，但 Alpha 的核心目标是让空间对象能被维护。

11.1 编辑对象

对象	几何	用途
POI	Point	传送点、商店、NPC、Boss 点、入口。
道路/铁路/水路	LineString / MultiLineString	导航、交通网络、路线规划。
建筑/城市/领地	Polygon / MultiPolygon	建筑边界、区域名、权限范围。
热力/活动区	Polygon / Raster overlay	活动密度、危险区、事件范围。

11.2 编辑能力

- **Snap:** 吸附到方块、chunk 边界、已有线端点、建筑角点。
- **Draft:** 普通玩家提交草稿，管理员或 cartographer 审核发布。
- **Revision:** 每个 feature 有版本历史，支持回滚和 diff。
- **Conflict:** 同一对象被多人编辑时锁定或进入 merge review。
- **Audit:** 记录 actor、时间、来源、修改前后内容。
- **Import/Export:** Cartography Feature JSON、CSV POI、插件数据源。

11.3 编辑器和底图的关系

编辑器永远在 raster basemap 上绘制 vector features。用户画道路、标建筑、写区域名时，不改底图 tile；底图只在世界方块变化或材质 profile 变化时重新渲染。

正确交互

用户画一条道路 -> 写入 `carto_features` -> vector tile cache invalidation -> 前端刷新道路图层。底图 raster tile 不需要重渲染。

错误交互

用户画一条道路 -> 把道路烤进 raster tile -> 之后改样式或权限时必须重渲染底图。这会破坏图层分离和权限模型。

12. API 和插件 SDK

API 不能过早冻结。v2.0 将 API 分为 internal、editor、plugin SDK 三层。

12.1 API 成熟度

层级	起始阶段	稳定性	用途
Internal API	Prototype/MVP	可破坏性变更。	渲染队列、snapshot、metatile、tileset 管理。
Editor API	Alpha	部分稳定，允许版本化迁移。	图层 CRUD、feature 编辑、审核、搜索。
Plugin SDK API	v1	冻结主干接口，语义版本化。	第三方插件写入图层、监听事件、导入服务器数据。

12.2 HTTP API

```
GET /api/cartography/v1/worlds
GET /api/cartography/v1/worlds/{world}/dimensions
GET /api/cartography/v1/tilegrids/{gridId}/manifest
GET /tiles/raster/{world}/{dimension}/{profile}/{tilesetVersion}/{z}/{x}/{y}.webp
GET /tiles/vector/{world}/{dimension}/{tilesetVersion}/{layerSet}/{z}/{x}/{y}.pbf

GET /api/cartography/v1/layers
POST /api/cartography/v1/layers
GET /api/cartography/v1/features?layer=roads&bbox=minX,minZ,maxX,maxZ
POST /api/cartography/v1/features
PATCH /api/cartography/v1/features/{featureId}
POST /api/cartography/v1/features/{featureId}/approve
GET /api/cartography/v1/features/{featureId}/history

GET /api/cartography/v1/live/players
WS /api/cartography/v1/live/events
```

12.3 Plugin SDK 示例

```
CartographyApi api = Cartography.getApi();

LayerHandle railLayer = api.layers().getOrCreate("rail_network", builder -> builder
    .world("survival_2026")
    .dimension("overworld")
    .geometryTypes(GeometryType.LINE_STRING, GeometryType.POINT)
    .permissionPolicy("public_transport_view")
);

railLayer.upsertFeature("rail_spawn_to_north", feature -> feature
    .lineString(List.of(
        Coord.xz(120, 80),
        Coord.xz(120, -400),
        Coord.xz(300, -900)
    ))
    .property("name", "主城北线")
    .property("class", "mainline")
    .weight(90)
);
```

12.4 插件不变量

- 插件写入 feature，不直接写 raster tile。
- 插件写入的坐标是 MC 原始坐标。
- 插件可以声明 layer/style/permission policy，但不能绕过权限过滤。
- 插件 SDK v1 才冻结；Alpha/Beta 的示例 API 只作为设计预览。

13. 存储和部署拓扑

Cartography 支持从单服本地到大型服务器的渐进式部署。关键是不要把在线写入、在线查询和静态归档混为一谈。

13.1 单服 MVP

```
Minecraft Server + Cartography Plugin
-> local render workers
-> filesystem tile store
-> SQLite feature store
-> embedded web server
```

组件	选择	理由
Raster store	filesystem	最简单，支持 atomic rename。
Feature store	SQLite	足够支撑 Alpha 前数据量。
Web	插件内嵌或 sidecar	低部署门槛。
Vector tiles	GeoJSON tile 或轻量 MVT	MVP 可延后。

13.2 中大型服务器

```
Minecraft Server
-> Cartography Adapter
-> Render Sidecar Workers
-> Object Storage / CDN for raster tiles
-> PostgreSQL / PostGIS for features
-> Tile API / Vector Tile Server
-> Web Editor
```

PostGIS 的 `ST_AsMVT` 可以从查询行生成 MVT 二进制表示，属性列可编码为 feature attributes [R6]。这适合作为专业部署的 vector tile 后端。

13.3 静态归档

赛季结束、建筑展览或公开下载时，可以把 raster/vector tile pyramid 打包为 PMTiles 或其他静态格式。PMTiles 是单文件瓦片金字塔，可通过 HTTP Range Requests 按需读取，但不适合在线高频更新 [R7]。

部署目标	推荐	不推荐
在线服务器地图	filesystem / object storage manifest + SQLite/PostGIS。	把 PMTiles 当主写入目标。
赛季归档	PMTiles + static hosting。	要求实时更新。
大型公共地图	CDN + immutable tilesetVersion。	同 URL 覆盖旧 tile。

14. 路线图: MVP 到完全完善

路线图按能力成熟度推进。每阶段都有明确的进入条件和退出条件，避免把愿景指标压到 MVP。

Stage 0 - Prototype

目标	范围	退出条件
验证正射渲染可行性。	读取 chunk snapshot、颜色表、单张 tile、基本 height shading。	能渲染指定区域 PNG/WebP；坐标转换单测通过。

Stage 1 - MVP

目标	范围	退出条件
可用底图系统。	top-down raster、dirty chunk、metatile、tile pyramid、玩家 marker、维度切换、TPS budget。	稳定浏览；block change 后最高 zoom 更新；父级 tile 有失效链；tile URL 版本化。

Stage 2 - Alpha

目标	范围	退出条件
空间对象可维护。	点线面 editor、layers、feature store、draft/approve、revision、basic search、permission hard filter。	玩家能提交道路/建筑/POI 草稿；管理员审核发布；权限测试通过。

Stage 3 - Beta

目标	范围	退出条件
现代 WebGIS 能力。	MVT、style document、POI ranking、label collision、Cartography Feature JSON 导入导出、PostGIS optional。	万级 feature 流畅；缩放显示合理；数据和样式分离稳定。

Stage 4 - v1 Production

目标	范围	退出条件
生产可用和生态接口。	Plugin SDK freeze、WorldGuard/Towny/claims/warps/rail integrations、对象存储发布、部署文档、migration。	第三方插件可接入；SDK 语义版本化；缓存发布模型生产验证。

Stage 5 - v2 Advanced

目标	范围	退出条件
高清和分析能力。	resource pack top texture、HD profile、computed layers、heatmap、route analytics、PMTiles archive。	高清离线渲染可控；静态归档可发布；计算图层可重建。

Stage 6 - v3 Complete Platform

目标	范围	退出条件
平台化 Cartography。	多服/多世界 portal、协作编辑治理、复杂权限、webhook、公开地图门户、可选斜视图 renderer。	多服务器地图资产统一管理；治理流程稳定；长期兼容承诺。

15. 测试、观测和验收

Cartography 的测试重点不是单纯渲染速度，而是坐标、缓存、失效、权限和更新链条是否可证明正确。

15.1 必须测试的不变量

测试	目的	失败表现
coordinate round-trip	证明 MC 坐标、view 坐标、signed tile、normalized tile 可互转。	Feature 和底图错位、负坐标错位。
metatile grouping	证明 dirty tile 会归并成 metatile job。	同一 metatile 被重复渲染、tile 缝隙。
ancestor invalidation	证明 maxZoom 更新会标记所有父级。	低 zoom 长期旧图。
permission hard filter	证明未授权 feature 不进候选集。	隐藏基地被聚类/排名泄露。
tilesetVersion cache	证明材质包/renderer/profile 改变会产生新 URL。	浏览器/CDN 拿旧图。
object manifest switch	证明对象存储发布不会出现半新半旧 manifest。	客户端读到不一致瓦片。
surface transparent structure	证明玻璃建筑不会被穿透消失。	温室/玻璃屋顶不可见。

15.2 观测指标

指标	说明	建议阈值/观察
snapshot_time_ms	主线程或同步路径拿快照耗时。	MVP 应持续低且可限流。
render_job_queue_depth	渲染任务积压。	长时间增长说明 budget 不足。
metatile_render_ms	单个 metatile 渲染耗时。	按 profile 分桶观察。
tile_write_latency_ms	写本地或对象存储耗时。	IO 压力导致延迟时应降速。
ancestor_dirty_backlog	低 zoom 父级积压。	可延迟但不能无限增长。
permission_filtered_count	权限过滤掉的 feature 数量。	用于安全审计和性能分析。
cache_hit_ratio	tile/API 缓存命中率。	低命中可能是版本策略或 URL 设计问题。

15.3 基准场景

- 小型建筑服: 3k x 3k overworld，玩家 20，普通建筑密度。
- 大型生存服: 20k x 20k overworld，玩家 100，WorldEdit/机器事件较多。

- 高 feature 场景: 100k POI/道路/建筑, 测试 MVT 和标签避让。
- 负坐标场景: 地图主体在 -10000,-10000 附近, 验证 normalized publish grid。
- 隐私场景: PVP/war mode, 验证公共地图不泄露玩家精确坐标。

16. 风险清单和决策点

风险	影响	缓解
modded block 渲染复杂	高清 profile 卡住。	MVP 颜色表 + fallback; v2 再做 resource pack top texture; 未知方块可配置。
坐标/瓦片模型返工	Feature 和底图错位。	v2.0 不变量先冻结; 所有阶段维护 coordinate round-trip 测试。
dirty 过度触发	TPS/CPU 压力。	MVP 保守但限流; Beta 加 surface-aware filter 和 batch merge。
权限泄露	隐藏基地、玩家位置泄露。	权限硬过滤; 隐私 preset; 安全测试进入验收。
缓存污染	用户看到旧图或混合图。	tilesetVersion 进入 URL; immutable cache; manifest switch。
SDK 过早冻结	生态插件被破坏性升级拖累。	v1 前只承诺 preview; v1 freeze 后语义版本化。
PMTiles 被误用于在线写入	发布链路不一致或性能问题。	文档和代码层限制: PMTiles builder 只走 archive/export pipeline。

16.1 仍需产品决策的问题

- **默认 serverMode:** 新安装时用 building、survival 还是 ask-first wizard。
- **公开地图策略:** 是否默认公开玩家位置, 建议默认不公开精确位置。
- **编辑权限:** 普通玩家能否直接发布 feature, 建议默认草稿+审核。
- **材质目标:** v1 是否必须有 resource pack top texture, 还是 v2 进入。
- **前端引擎:** Alpha 用 OpenLayers 快速落地, Beta 是否切 MapLibre/MVT 样式体系。

附录 A. 配置、伪代码和协议示例

A.1 cartography.yml

```
server:
  mode: survival
  publicMap: true

coordinates:
  crs: cartography:mc-crs
  dataCoordinate: minecraft-xz
  tileCoordinateMode: online-signed
  tileOrigin:
    x: 0
    z: 0

render:
  enabled: true
  workers: auto-minus-1
  maxSnapshotsPerTick: 2
  pauseBelowTps: 18.5
  resumeAboveTps: 19.2
  metatileSize: 4
  paddingBlocks: 2
  dirtyDebounceSeconds: 5

profiles:
  fast:
    pixelsPerBlockAtMaxZoom: 2
    format: webp
    quality: 85
    materialMode: color-table
    heightShade: true
    softShadow: false
  vanilla:
    pixelsPerBlockAtMaxZoom: 4
    materialMode: top-texture
    heightShade: true
    fluidDepth: true

privacy:
  players:
    public: blurred-region
    friends: exact-if-authorized
    admins: exact
    publicDelaySeconds: 60

cache:
  tileUrlIncludesTilesetVersion: true
  immutablePublishedTiles: true
  manifestCacheControl: no-cache

storage:
  raster:
    type: filesystem
    path: ./plugins/Cartography/tiles
  features:
    type: sqlite
    path: ./plugins/Cartography/cartography.db
```

A.2 Tile manifest

```
{
  "tilesetVersion": "a18f03c99f2e10b7",
  "world": "survival_2026",
  "dimension": "overworld",
  "profile": "fast",
  "resourcePackHash": "vanilla-1.21.6-abc123",
  "rendererVersion": "0.1.0",
  "materialTableHash": "mats-f0a01d",
  "tileGrid": {
    "mode": "published-normalized",
    "tileSize": 256,
    "maxZoom": 8,
    "minZoom": 0,
    "pixelsPerBlockAtMaxZoom": 2,
    "tileOriginX": 0,
    "tileOriginZ": 0,
    "minSignedTileX": -183,
    "minSignedTileY": -96,
    "normalizedOffsetX": 183,
    "normalizedOffsetY": 96
  }
}
```

A.3 Dimension surface rules

```
dimensions:
  overworld:
    surfaceMode: highest_visible_surface
    preserveTransparentStructure: true
    fluidDepth: true
  nether_roof:
    surfaceMode: highest_visible_surface
    includeBedrockRoof: true
  nether_caves:
    surfaceMode: cave_surface
    ignoreAboveY: 120
    ignoreBlocks:
      - minecraft:bedrock
  the_end:
    surfaceMode: highest_visible_surface
```


A.4 数据流依赖规则

依赖	允许	禁止
Renderer -> Feature Store	只允许读取 style profile 中的视觉参数，不读业务 feature。	渲染器内置道路/领地/交通网络语义。
Feature Store -> Raster Cache	只有当 feature 被明确设置为 baked overlay profile 时才触发专用底图重渲染，默认不允许。	普通道路编辑导致底图重渲染。
Permissions -> Feature Query	硬过滤。	作为 ranking penalty。
Tile Store -> Render Jobs	读取已有 child tiles 进行 downsample。	tile_cache 作为 render_jobs 的上游业务事实源。

附录 B. 参考资料

以下参考资料用于约束本文中的 API、格式和语义判断。Cartography 的 MC CRS 是自定义坐标语义，因此即使使用 GeoJSON/MVT 结构，也必须通过 manifest 或 metadata 明确坐标解释。

1. **[R1] PaperMC ChunkSnapshot JavaDoc.** ChunkSnapshot 是静态、线程安全的 chunk 快照，设计目的包括把 chunk 数据复制出来交给另一个线程处理，例如地图渲染。
<https://jd.papermc.io/paper/org/bukkit/ChunkSnapshot.html>
2. **[R2] PaperMC World JavaDoc - getChunkAtAsync.** 异步 chunk 加载 API 不保证立刻返回，适合把 chunk 请求从即时主线程路径中拆开。
<https://jd.papermc.io/paper/1.16.5/org/bukkit/World.html>
3. **[R3] RFC 7946 - The GeoJSON Format.** GeoJSON 采用 WGS 84 和十进制度数语义，因此 Minecraft X/Z 不能直接声称标准 GeoJSON 地理语义。
<https://www.rfc-editor.org/info/rfc7946/>
4. **[R4] Mapbox Vector Tile Specification.** MVT 内部几何使用 tile 局部网格坐标，不自带真实地理坐标语义。
<https://mapbox.github.io/vector-tile-spec/>
5. **[R5] MapLibre Style Spec.** Style 的 sources 与 layers 决定哪些地图要素显示以及如何显示。
<https://maplibre.org/maplibre-style-spec/>
6. **[R6] PostGIS ST_AsMVT.** ST_AsMVT 从行集合生成 Mapbox Vector Tile 二进制表示，属性列可编码为 feature attributes。
https://postgis.net/docs/ST_AsMVT.html
7. **[R7] PMTiles Concepts.** PMTiles 是单文件瓦片金字塔，可通过 HTTP Range Requests 按需读取，但为 read-only，不能原地事务更新。
<https://docs.protomaps.com/pmtiles/>
8. **[R8] OpenLayers Projection API.** OpenLayers Projection 支持定义/注册自定义投影、extent 与 units 等。
https://openlayers.org/en/latest/apidoc/module-ol_proj_Projection-Projection.html

结语

Cartography v2.0 的核心修正就是把“愿景完整”推进到“工程闭环”。底图、图层、坐标、缓存、权限、发布和 SDK 都有明确边界：底图负责外观，Feature 负责语义，权限负责候选集，ranking 负责显示优先级，tilesetVersion 负责缓存命名空间，metatile 和祖先失效负责更新一致性。

按这个版本推进，MVP 不会被产品愿景拖垮，后续 Alpha/Beta/v1 又能自然扩展成完整的 Minecraft WebGIS 平台。